

```

let iRight = 0;
while (sortedResult.length < items.length) {
    // Append the smaller value to sortedResult.
    if (left[iLeft] < right[iRight]) { ❹
        sortedResult.push(left[iLeft]);
        iLeft++;
    } else {
        sortedResult.push(right[iRight]);
        iRight++;
    }

    // If one of the pointers has reached the end of its list,
    // put the rest of the other list into sortedResult.
    if (iLeft == left.length) {
        Array.prototype.push.apply(sortedResult, right.slice(iRight));
        break;
    } else if (iRight == right.length) {
        Array.prototype.push.apply(sortedResult, left.slice(iLeft));
        break;
    }
}

document.write("<pre>The two halves merged into: [" + sortedResult.join(", ")
+
"]</pre>");

return sortedResult; // Returns a sorted version of items.
}

let myList = [2, 9, 8, 5, 3, 4, 7, 6];
myList = mergeSort(myList);
document.write("<pre>[" + myList.join(", ") + "]</pre>");
</script>

```

The `mergeSort()` function (and all the recursive calls to the `mergeSort()` function) takes an unsorted list and returns a sorted list. The first step in this function is to check for the base case of a list containing only zero or one item ❶. This list is already sorted, so the function returns the list as is.

Otherwise, the function determines the middle index of the list ❷ so that we know where to split it into the left- and right-half lists to pass to two recursive function calls ❸. The recursive function calls return sorted lists, which we store in the left and right variables.